LEVEL 4 — NumPy Data Types

Easy line-by-line notes with examples

1. Integer (int)

Integer stores whole numbers without decimal points.
Example:

```
import numpy as np
arr = np.array([10, 20, 30], dtype=np.int32)
print(arr)
```

Output: `[10 20 30]`

```
print(arr.dtype)
```

Output: `int32`

2. Unsigned Integer (uint)

Unsigned integers store only non-negative whole numbers (0 and positive values).
Example:

```
arr = np.array([0, 10, 20], dtype=np.uint8)
print(arr)
```

Output: `[ 0 10 20]`

```
print(arr.dtype)
```

Output: `uint8`

3. Float

Float stores numbers with decimal values.
Example:

```
arr = np.array([10.5, 20.2, 30.7], dtype=np.float64)
print(arr)
```

Output: `[10.5 20.2 30.7]`

4. Complex

Complex numbers contain a real part and an imaginary part.
Example:

```
arr = np.array([2+3j, 4+5j])
print(arr)
```

Output: `[2.+3.j 4.+5.j]`

5. Boolean

Boolean stores True or False values.
Example:

```
arr = np.array([True, False, True])
print(arr)
```

Output: `[ True False True]`

```
print(arr.dtype)
```

Output: `bool`

6. String

String stores text. NumPy can create a fixed-width Unicode string dtype.
Example:
```
arr = np.array(["Apple", "Mango", "Banana"])
print(arr)
```
Output: `['Apple' 'Mango' 'Banana']`

7. Unicode

Unicode allows text characters from many languages and symbols. In modern NumPy, ordinary Python strings commonly become Unicode string dtypes such as `<U5`.
Example:
```
arr = np.array(["Hello", "■■■■■■"])
print(arr)
```

8. Object

Object dtype can store Python objects of different kinds.
Example:
```
arr = np.array([10, "Hello", 3.5], dtype=object)
print(arr)
```
Output: `[10 'Hello' 3.5]`

9. Checking dtype

Use `.dtype` to check the data type of an array.
Example:
```
arr = np.array([10, 20, 30])
print(arr.dtype)
```
Typical output: `int64` (the exact default integer size can depend on the platform).

10. Changing dtype

You can create a new array with another dtype using `astype()`.
Example:
```
arr = np.array([10, 20, 30])
new_arr = arr.astype(np.float32)
print(new_arr)
```
Output: `[10. 20. 30.]`
```
print(new_arr.dtype)
```
Output: `float32`

11. astype()

`astype()` converts an array to another data type and returns a new array.
Example:
```
arr = np.array([1.2, 2.5, 3.9])
new_arr = arr.astype(int)
print(new_arr)
```
Output: `[1 2 3]`
Decimal parts are removed when converting these positive floats to integers.

12. Type Conversion

Type conversion means changing values from one data type to another. NumPy commonly uses `astype()` for this.

```
arr = np.array([1, 2, 3])
arr_float = arr.astype(float)
print(arr_float)
```

Output: `[1. 2. 3.]`

13. Memory Optimization

Smaller numeric dtypes can use less memory when their value range is sufficient. For example, int32 uses less memory per element than int64.

```
arr = np.array([10, 20, 30], dtype=np.int64)
small_arr = arr.astype(np.int32)
print(arr.nbytes)
print(small_arr.nbytes)
```

Important: choose a dtype only when it can safely hold all required values. For example, uint8 can store 0–255, while int8 stores -128–127.

Quick Revision

int → whole numbers
uint → non-negative whole numbers
float → decimal numbers
complex → real + imaginary numbers
bool → True / False
string / Unicode → text
object → Python objects
dtype → checks data type
astype() → converts data type
nbytes → checks memory used by array